Министерство образования и науки
Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
СИСТЕМ УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра конструирования узлов и деталей радиоэлектронной аппаратуры
(КУДР)

А.А. Бомбизов, А.Г. Лощилов, А.К. Пащенко

РАБОТА С ПОРТАМИ ВВОДА-ВЫВОДА. ОРГАНИЗАЦИЯ ВВОДА ИНФОРМАЦИИ

Методические указания к выполнению
лабораторной и самостоятельной работы
по дисциплине «Программирование микроконтроллеров»

Томск 2026

1 Введение

В предыдущей работе была освоена среда программирования Arduino IDE и получены навыки организации вывода цифровых сигналов через порт ввода-вывода общего назначения.

Целью настоящей работы является освоение приемов ввода информации в микроконтроллер посредством аппаратных и программных средств. В лабораторной работе будет рассмотрено программирование портов ввода-вывод для чтения сигналов и настройка последовательного порта, обеспечивающего обмен информацией с ПК.

2 Чтение сигналов с портов ввода-вывода

С внешним миром микроконтроллер взаимодействует через порты ввода-вывода [1, п. 14]. Рисунок 1 изображает структурную схему одного порта.

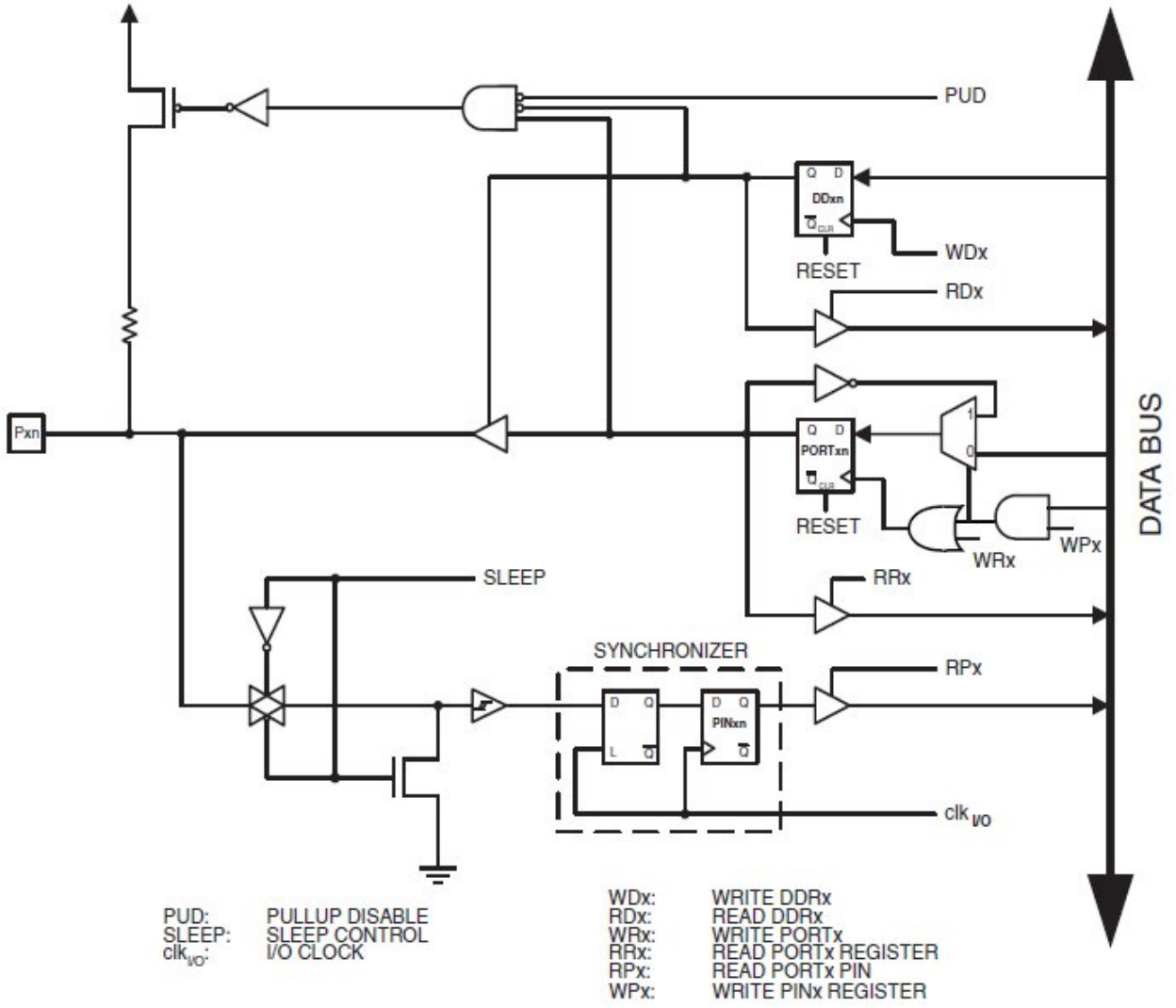


Рисунок 1 – Структурная схема одного вывода порта ввода–вывода микроконтроллера

Согласно приведенному рисунку, выходной контакт изображен в квадрате с подписью Pxn, где x – буква порта, а n – номер вывода порта. За

управление портами отвечают три регистра (регистр процессора, биты (флаги) которого отражают состояние процессора и свойства результатов предыдущих операций): PORTx, PINx, DDRx. К примеру, за порт «B» отвечают три восьмиразрядных регистра PORTB, PINB, DDRB, а каждый бит в этих регистрах отвечает за соответствующий вывод порта. Это как линейка из восьми переключателей. DDRx, PORTx были рассмотрены в предыдущей работе.

PINx – регистр чтения. Из него можно только читать. В регистре PINx содержится информация о реальном текущем логическом уровне на выводах порта вне зависимости от настроек порта. Таким образом, для определения состояния вывода достаточно только прочитать значение соответствующего бита регистра.

Рисунок 2 изображает схему замещения, которая дает более наглядное представление о логике работы порта ввода-вывода1.

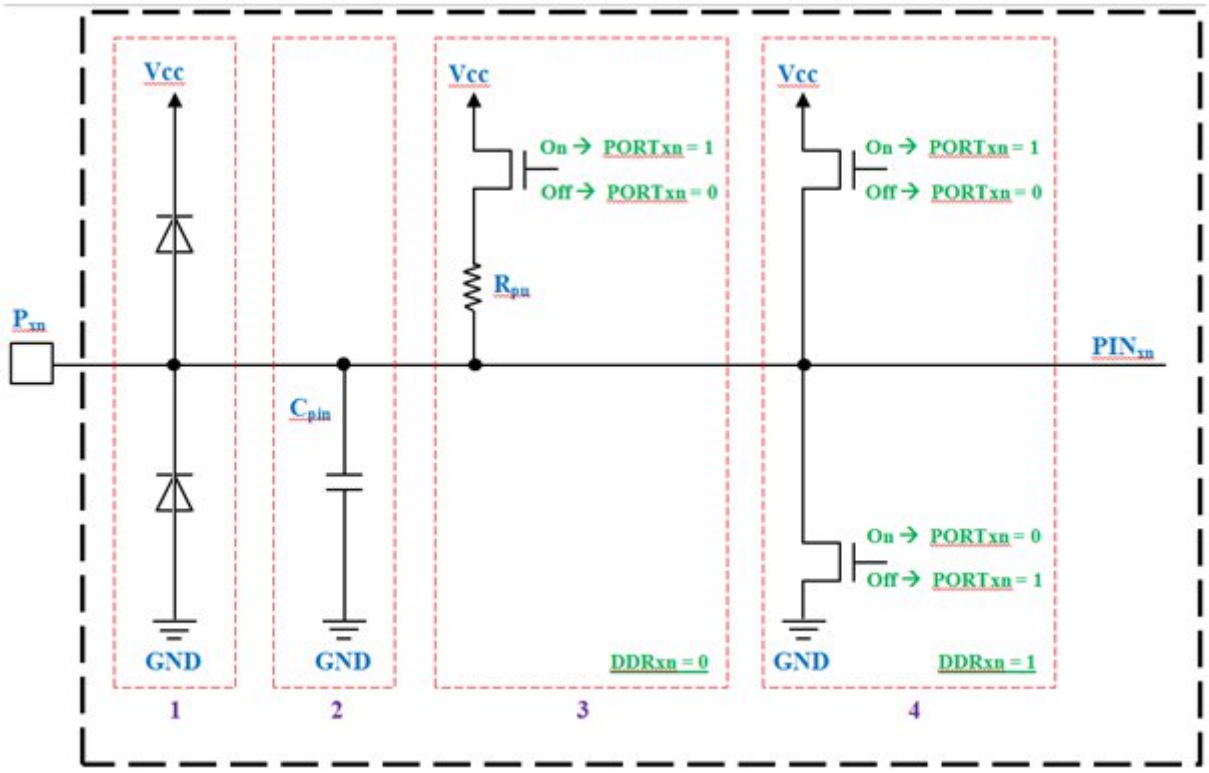


Рисунок 2 – Схема замещения порта ввода–вывода микроконтроллера

Pxn – имя вывода порта микроконтроллера, где x буква порта (A, B, C или D), n номер разряда порта (7… 0);

Cpin – паразитная емкость порта;

VCC – напряжение питания;

Rpu – отключаемый нагрузочный резистор (pull-up);

PORTxn – бит n регистра PORTx; PINxn – бит n регистра PINx.

DDRxn – бит n регистра DDRx.

Всю схему замещения можно разделить на четыре блока:

1) Защита вывода микроконтроллера от кратковременных импульсов перенапряжения, выполненная с использованием двух диодов. Если напряжение будет выше питания, то верхний диод откроется, и это напряжение будет отводиться на шину питания. Если на вывод попадет отрицательное (ниже нулевого уровня) напряжение, то оно будет нейтрализовано через нижний диод, подключенный на «землю».

2) Конденсатор – это паразитная емкость вывода. Из-за малых размеров она, как правило, не берется в расчет.

3) Блок включения подтягивающего резистора (pull-up), включенный между проводником, по которому распространяется электрический сигнал, и питанием, либо между проводником и землей. Подтягивающий резистор нужен, чтобы гарантировать на логическом входе, с которым соединен проводник, высокий (в первом случае) либо низкий (во втором случае) уровень в случаях:

- проводник не соединен с логическим выходом;
- присоединённый логический выход находится в высокоимпедансном состоянии.
- когда разомкнут ключевой элемент на присоединённом логическом выходе, который устроен, как открытый вывод ключевого элемента. Ключевым элементом может быть полевой или биполярный транзистор, а открытым выводом — коллектор, эмиттер, сток или исток. А совсем в простом случае ключевым элементом может быть кнопка.

За подключение подтягивающего резистора согласно структурной схеме (Рисунок 1) отвечают два регистра, управляющих логическим элементом «И». Это PORTxn и бит PUD в регистре MCUCR. По умолчанию бит PUD – Pull-up Disable – выставлен в логический ноль, поэтому для активации подтягивающего резистора никаких дополнительных манипуляций с этим битом не требуется выполнять.

4) Блок управления выходом. Эта часть работы была рассмотрена в предыдущей работе.

Для выполнения операций с помощью средств Arduino IDE с выводами порта, настроенными на вход, необходимо использовать функции *pinMode()*, *digitalRead()*.

Далее представлен пример использования данных функций

```
int led1=13;
//A1 – константа, соответствующая в среде Arduino IDE кнопке S1
int button1=A1;
int val1 = 0;
void setup()
```

```
{
        pinMode(led1, OUTPUT);  // настройка led1 на выход
        pinMode(button1, INPUT); // настройка button1 на вход
}
void loop()
{
        // выполнение чтения  в переменную val1 состояния кнопки S1
        val1= digitalRead(button1);
         digitalWrite(led1,val1);    // управление светодиодом D1
        delay(250);                 // задержка 250 мс
}
```

На практике достаточно часто работа с микроконтроллером осуществляется путём прямого обращения к регистрам. Физическое изменение состояния на входе микроконтроллера отражается в изменении состояния соответствующего бита в регистре PINx. Рисунок 3 изображает битовую схему регистра PINB.

Bit	7	6	5	4	3	2	1	0
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

Рисунок 3 – Битовая схема регистра

Согласно битовой схеме каждый бит с наименованиями PINB0–PINB7 соответствуют выводам микроконтроллера PB0–PB7. Рисунок 4 изображает расположение выводов.

Как было описано в прошлой работе, переключение вывода на выход или на вход осуществляется путём установки соответствующего бита в регистре в 1 или 0, соответственно. Например, для переключения вывода PB4 на выход необходимо выполнить следующую операцию:

```
DDRB|=(1<<4);
```

то есть выполнение операции побитового ИЛИ значения в регистре с маской BIN:00010000.

Переключение вывода PB4 на вход достаточно выполнить операцию:

```
DDRB&=~(1<<4);
```

где & – операция побитового И, ~ – инверсия, которая приводит маску к виду BIN:11101111.

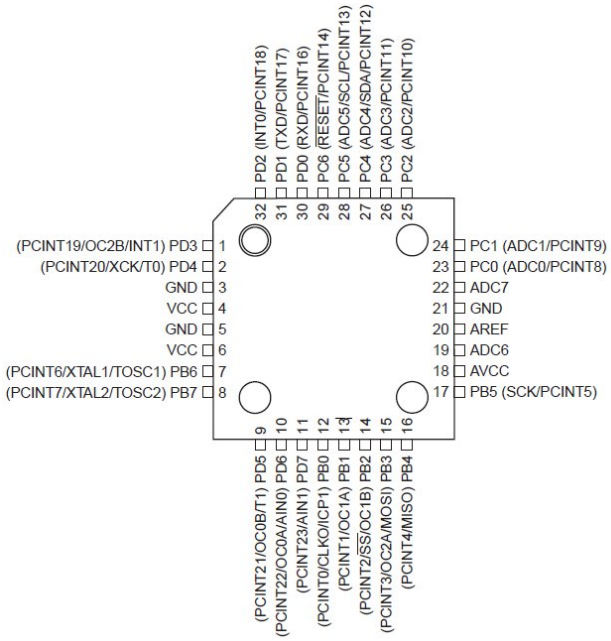


Рисунок 4 – Расположение выводов (цоколёвка – pinout)

В программе определение значения бита регистре может быть выполнена двумя способами:

1) *val1=(PINB&(1<<4))>>4;*

где *val1* – предварительно объявленная целочисленная переменная. А действия справа от «=» трактуются следующим образом:

а) рассчитывается маска (*1<<4),* в результате чего получится число вида BIN:00010000;

б) выполняется операция побитового И со значением регистра PINB: BIN:XXXXXXXX&00010000=000X0000. В результате будет получено число, четвертый бит которого будет соответствовать значению на выводе микроконтроллера;

в) выполнение сдвига вправо на 4. BIN: 000X0000 >> 4 = 0000000X. В результате последней операции будет получено число 0 или 1, которое в дальнейшем может быть использовано в программе.

2)

```
if((PINB&(1<<4)))
{
//то на входе единица
}
else
{
//иначе на входе ноль
}
```

3 Взаимодействие с последовательным портом

Последовательный порт или COM-порт (произносится "ком-порт", от англ. COMmunication port) – двунаправленный последовательный интерфейс, предназначенный для обмена байтовой информацией.

Последовательная передача данных является наиболее распространенным низкоуровневым протоколом обмена информацией между устройствами. Как следует из названия, последовательный порт отправляет и получает байты информации последовательно, один бит за другим.

Последовательный порт получил широкое распространение для установления связи с различными аппаратными средствами встраиваемых вычислительных систем.

Для реализации работы с последовательным портом, в среде Arduino IDE представлен набор функций Serial, который служит для связи устройства Arduino с компьютером или другими устройствами, поддерживающими последовательный интерфейс обмена данными.

Инициализация последовательного порта (обычно в процедуре setup) выполняется при помощи функции

Serial.begin(*speed*)

где *speed* – скорость передачи данных, бит/с.

Для обмена данных используются следующие скорости: 900, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, 115200 и др. Наиболее часто используют скорости 9600 или 115200.

Отправка данных в последовательный порт

Оправка сообщения выполняется с помощью функций:

Serial.print(*val*) – вывод значения в виде текстовой строки;

Serial.print(*val*, *format*) – вывод значения в виде текстовой строки, с указанием формы вывода;

Serial.println(*val*) – вывод текстовой строки с символом переноса;

Serial.println(*val*, *format*) – форматированный вывод текстовой строки с символом переноса.

где *val* – значение, может быть как числовым (целым или вещественным), так и текстовым: Serial.print(42); Serial.print("Hello");

format может принимать следующие значения:

BIN – вывод двоичного числа;

OCT – вывод восьмеричного числа;

DEC – вывод десятиричного числа;

HEX – вывод шестнадцатеричного числа;

числовое значение – количество знакомест отведенных для вывода вещественной части *val*.

Для вывода байта или последовательности байт в неизменном виде, можно использовать функцию Serial.write( ).

Возможные варианты использования:

Serial.write(*val*) – вывод байта;

Serial.write(*str*) – вывод строки в виде последовательности байт;

Serial.write(*buf*, *len*) – вывод массива, где *len* – длинна массива.

Чтение данных из последовательного порта

Для чтения информации из последовательного порта, необходимо сначала выполнить проверку наличия вновь поступивших данных в буфер порта. Обычно это выполняется путем циклического вызова функции Serial.available().

Данная функция возвращает количество байт (символов) доступных для чтения из последовательного интерфейса связи. Это те байты которые уже поступили и записаны в буфер последовательного порта. Буфер может хранить до 64 байт.

Для чтения очередного доступного байта из буфера последовательного порта применяется функция Serial.read().

В качестве примера, ниже приведен один из возможных вариантов чтения данных из последовательного порта с отправкой этих же данных назад:

```
int k;
while(Serial.available()) // циклическая проверка наличия данных
{
    k = Serial.read();      // чтение байта из последовательного порта
    Serial.write(k);        // отправка байта в последовательный порт
}
```

Чтение и отправка данных в среде Arduino IDE

Среда разработки Arduino имеет встроенный монитор последовательного интерфейса (Serial monitor). Для начала обмена данными необходимо запустить монитор нажатием кнопки Монитор порта. В появившемся окне установить скорость связи. Рисунок 5 изображает описанный алгоритм запуска окна монитор последовательного интерфейса.

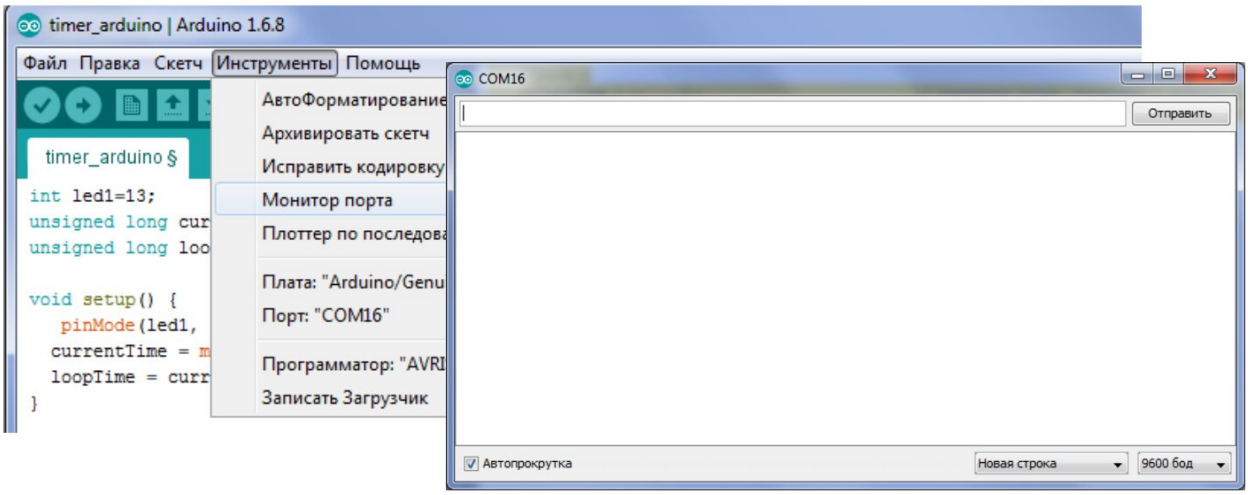


Рисунок 5 – Монитор порта

4 Порядок выполнения работы

В практической части работы должны быть решены две задачи: освоение работы с портами ввода-вывода, настроенными на режим входа; закрепление полученных навыков при решении задачи счетчика (значение счетчика должно отображаться в двоичной форме на светодиодах; увеличение и уменьшение значения выполняется кнопками S1–S2; сброс счетчика выполняется кнопкой S3). Выполнение работы должно быть выполнено в двух вариантах: управление режимами и состояниями выводов путём непосредственного обращения к регистрам микроконтроллера; управление с помощью функций Arduino IDE. Для этого необходимо выполнить следующие действия:

4.1 Изучите предложенный в п. 2 и 3 теоретический материал.

4.2 Подключите отладочную плату к компьютеру как описано в теоретической части.

4.3 Создайте новый проект (меню Файл->Новый) и сохраните его в свой рабочий каталог.

Освоение работы с портами ввода-вывода, настроенными на режим входа:

4.4 С использованием функций Arduino IDE реализуйте алгоритм переключения светодиода по нажатию кнопки.

4.5 Реализуйте алгоритм, выполненный в предыдущем пункте, но только переключение светодиода должно происходить **однократно** по отпусканию кнопки или по нажатию.

4.6 Заведите в программе переменную для хранения значения 4х-битного счетчика.

4.7 Реализуйте алгоритм для отображения значения счетчика на светодиодах отладочного комплекта и вывода в последовательный порт.

4.8 Определите по схеме [2] наименования выводов микроконтроллера, соответствующих кнопкам S1–S3.

4.9 Реализуйте алгоритмы увеличения, уменьшения и сброса значения счетчика при нажатии соответствующих кнопок.

4.10 Создайте новый проект и реализуйте алгоритмы, выполненные в пунктах 4.5 – 4.9, но только с использованием прямого обращения к регистрам микроконтроллера.

4.11 Оформите отчет, содержащий титульный лист и разделы: введение, ход выполнения работы, ответы на контрольные вопросы и выводы.

4.12 Защитите отчет у преподавателя.

5 Контрольные вопросы

5.1 Каким образом переключить вывод микроконтроллера на вход?

5.2 Каково назначение подтягивающего резистора?

5.3 Сколько выводов у микроконтроллера, с которыми можно работать как с портами ввода вывода?

5.4 Какой регистр отвечает за состояние на выводе микроконтроллера?

5.5 Какая функция отвечает за скорость обмена данными по последовательному порту?

5.6 Каким образом производится чтение данных из последовательного порта?

Список литературы

1 ATMEL 8-BIT MICROCONTROLLER WITH 4/8/16/32KBYTES, IN-SYSTEM PROGRAMMABLE FLASH.– URL: http:\\www.atmel.com\images\Atmel-8271-8-bit-AVR-Microcontroller-ATmega48A-48PA-88A-88PA-168A-168PA-328-328P_datasheet_Complete.pdf (дата обращения: 10.01.2017);

2 UNO Schematic – Arduino.– URL: https:\\www.arduino.cc\en\uploads\Main\arduino-uno-schematic.pdf (дата обращения: 10.01.2017);